

AUTOMATION

Launch sessions from links

Open a Claude Code terminal session from a URL. Embed `claude-cli://` links in runbooks, alerts, and dashboards so a click opens Claude Code in the right repo with the right prompt.

A deep link is a `claude-cli://` URL that opens Claude Code in a new terminal window. The URL can carry a working directory and a prompt to pre-fill.

This lets you share a one-click starting point for a task: anyone with Claude Code installed who clicks the link sees a session open with the prompt already typed. The prompt is populated but not sent until you press Enter.

Because a deep link is a URL, you can put one anywhere a link can go:

- An incident runbook step that opens the affected service's repo with a diagnostic prompt
- A monitoring alert or dashboard that links to an investigation prompt for a specific metric
- A README or wiki page that opens the project with an onboarding prompt
- A CI failure notification that pre-fills the failing job's name

This page covers how to **build a link**, **embed one in a runbook or trigger it from the shell**, and **manage or disable handler registration** on each platform.

 Deep links require Claude Code v2.1.91 or later.

How it works

The `claude-cli://` prefix is a custom URL scheme that Claude Code registers with your operating system, similar to how `mailto:` links open your email client. The link can live on a web page, in a wiki, in a Slack message, or in any app that renders links. When you click one:

1. The browser or app hands the URL to your operating system.
2. The operating system recognizes the `claude-cli://` prefix and starts Claude Code on your machine.

3. A new terminal window opens with Claude Code running in the directory the link specified, and the link's prompt text already in the input box.
4. You read the prompt, edit it if you want, and press Enter to send it.

The link itself can be hosted anywhere, but the session always opens locally on the computer where you clicked. See [Registration and supported platforms](#) for which terminal emulator opens on each operating system.

❗ The platform that displays the link must allow custom URL schemes. GitHub-rendered Markdown allows `http` and `https` but strips schemes like `claude-cli://` in READMEs, issues, pull requests, and wikis. Only the link text shows, with no link behind it and the URL hidden. See [Troubleshooting](#) for a workaround.

What a launched session shows

A deep link never executes anything on its own. The link only chooses a directory and fills the prompt box. If you click a link from a page you do not trust, the prompt is still inert: nothing reaches the model until you read what was filled in and press Enter.

When the session opens, a warning line below the input box reads `Prompt from an external link` and stays visible until you send or clear the prompt. For prompts over 1,000 characters, the warning includes the character count and tells you to scroll and review the full text before pressing Enter, since long prompts can push instructions off screen. Permission rules, `CLAUDE.md`, and trust prompts for the selected directory apply the same way as for any other session.

Build a link

Every deep link starts with `claude-cli://open`, which is the only path the handler accepts, followed by optional query parameters. The minimal form opens Claude Code in your home directory with an empty prompt:

```
claude-cli://open
```

Add parameters to control where the session starts and what the prompt box contains:

Parameter	Description
<code>q</code>	Text to pre-fill in the prompt box. URL-encode the value. Use <code>%0A</code> for line breaks in multi-

Parameter	Description
	line prompts. Maximum 5,000 characters.
<code>cwd</code>	Absolute path to use as the working directory. Network and UNC paths are rejected.
<code>repo</code>	A GitHub <code>owner/name</code> slug. Claude Code resolves it to a local clone it has seen before and starts there. If you have no matching clone, the session opens in your home directory instead.

`cwd` and `repo` are **two ways to set the working directory**. If you pass both, `cwd` takes precedence and `repo` is ignored, even if the `cwd` path does not exist.

The following link points at a repository called `acme/payments` with a two-line diagnostic prompt. Replace `acme/payments` with your repository's `owner/name` slug when you build your own:

```
claude-cli://open?
repo=acme/payments&q=Investigate%20the%20failed%20deploy%20of%20p
ayments-
api.%0ACheck%20recent%20commits%20to%20main%20and%20the%20last%20
successful%20build.
```

Clicking it opens a new terminal window, starts Claude Code in your local clone of `acme/payments` , and fills the prompt box with the decoded text:

```
Investigate the failed deploy of payments-api.
Check recent commits to main and the last successful build.
```

You can edit the prompt before pressing Enter to send it. If you have no local clone of the repository, the session opens in your home directory instead. See **Choose between `cwd` and `repo`** for how the local path is selected when you have multiple clones or worktrees.

Choose between `cwd` and `repo`

Use `cwd` when everyone who clicks the link has the project at the same absolute path, such as a standardized devcontainer or VM image.

Use `repo` when the link is shared and each person clones to a different location. Claude Code resolves the slug to a local path as follows:

- Each time you run `claude` in a Git repository, that directory's filesystem path is recorded

against the repository's GitHub `owner/name slug`.

- When a deep link arrives, `repo` opens whichever matching path you used most recently. Multiple clones and worktrees are tracked separately, so it picks the one you worked in last.
- The lookup only finds paths where you have already run Claude Code at least once.
- The link does not change which branch is checked out. The session opens in whatever state that directory is currently in.

The welcome header shows which path it picked so you can confirm the right clone opened.

Examples

The sections below show two common ways to use a deep link: as a Markdown link in a document and as a command in a script or shell alias.

Embed a link in a runbook

A deep link in a runbook gives whoever is triaging a one-click way to start investigating in the right repository with a prepared prompt. The platform that renders the runbook must allow custom URL schemes. GitHub-rendered Markdown does not allow `claude-cli://`, so a deep link in a GitHub README, issue, or wiki shows only its label with no clickable link. See [the troubleshooting note](#) for a workaround.

The prompt is part of the URL and must be URL-encoded. To produce the encoded value, pass your prompt text through `encodeURIComponent` in a browser console or any URL encoder.

The example below adds an investigation entry point to an incident runbook for a service called `web-gateway` :

```
## High 5xx rate on web-gateway
```

1. Acknowledge the page in PagerDuty.
2. [Open Claude Code in the gateway repo](claude-cli://open?repo=acme/web-gateway&q=5xx%20rate%20is%20elevated%20on%20web-gateway.%20Check%20recent%20deploys%2C%20error%20logs%20from%20the%20last%2030%20minutes%2C%20and%20open%20incidents%20in%20Linear.
.)
3. Post initial findings in #incident.

To use this in your own runbook, replace `acme/web-gateway` with your service's repository slug. This allows engineers with Claude Code installed and a local clone of that repository to click step 2 and start investigating with the prompt ready to send.

Open a link from the shell

You can also open a deep link from a shell script, alias, or automation rather than by clicking it. Call your operating system's URL-opening command with the link as the argument.

macOS

The built-in `open` command passes the URL to the registered `claude-cli://` handler:

```
open "claude-cli://open?repo=acme/payments&q=review%20open%20PRs"
```

Linux

Most desktop environments provide `xdg-open`, which passes the URL to the registered handler:

```
xdg-open "claude-cli://open?  
repo=acme/payments&q=review%20open%20PRs"
```

Windows

In PowerShell, `Start-Process` passes the URL to the registered handler:

```
Start-Process "claude-cli://open?  
repo=acme/payments&q=review%20open%20PRs"
```

In `cmd.exe`, `start` treats its first quoted argument as a window title, so pass an empty title before the URL:

```
start "" "claude-cli://open?
repo=acme/payments&q=review%20open%20PRs"
```

Registration and supported platforms

Claude Code registers the `claude-cli://` handler with your operating system the first time you start an interactive session on macOS, Linux, and Windows. You do not run a separate install command. Registration writes to user-level locations only:

Platform	Handler location
macOS	<code>~/Applications/Claude Code URL Handler.app</code>
Linux	<code>claude-code-url-handler.desktop</code> under <code>\$XDG_DATA_HOME/applications</code> , defaulting to <code>~/.local/share/applications</code>
Windows	<code>HKEY_CURRENT_USER\Software\Classes\claude-cli</code>

The handler launches Claude Code in a detected terminal emulator. On macOS, Claude Code remembers the terminal from your most recent interactive session and reuses it, supporting iTerm2, Ghostty, kitty, Alacritty, WezTerm, and Terminal.app. On Linux it honors the `$TERMINAL` environment variable, then `x-terminal-emulator` , then a list of common emulators. On Windows it prefers Windows Terminal, then PowerShell, then `cmd.exe` .

To prevent registration entirely, set `disableDeepLinkRegistration` to "disable" in `settings.json` . To enforce this across an organization so users cannot re-enable it, set it in managed settings instead.

Open a VS Code tab instead of a terminal

The VS Code extension registers its own handler at `vscode://anthropic.claude-code/open` , which opens a Claude Code editor tab rather than a terminal window. See Launch a VS Code tab from other tools for that URL's parameters.

Troubleshooting

Clicking the link does nothing

The handler likely is not registered yet. Start an interactive `claude` session once on that machine, exit, and try the link again. If you are on Linux without a desktop environment, `xdg-open` may have nothing to dispatch to.

The link renders as plain text instead of being clickable

Some Markdown renderers only allow `http` and `https` links and strip other URL schemes. GitHub does this in READMEs, issues, pull requests, and wikis: `[label](claude-cli://...)` renders as just `label`, with no link and the URL removed. On these platforms, put the deep link in a code block so readers can see the URL and paste it into their browser's address bar.

The session opens in my home directory instead of the repo

The `repo` parameter only resolves to clones Claude Code has already seen. Run `claude` inside the clone once so its path is recorded, or switch the link to use `cwd` with an absolute path.

The link opens the wrong terminal

On macOS, start `claude` in your preferred terminal once and the next deep link will use it. On Linux, set the `$TERMINAL` environment variable to your preferred emulator's command name. On Windows, the order is fixed: install Windows Terminal if you want links to open there instead of a PowerShell or `cmd.exe` window.

Learn more

These pages cover related ways to launch or extend Claude Code sessions:

- **Skills**: store a long runbook prompt as a `/skill` in the repo so the deep link's `q` parameter only has to name it
- **Non-interactive mode**: run Claude from a script and capture the output without opening a terminal